

PCMI: Persistent Cognitive Memory Infrastructure

A Scalable, Durable, and Multi-Tenant Memory Layer
for Long-Lived AI Agents

Marco Spagnuolo

<https://github.com/marco-spagn/pcmi>

Technical Architecture Report v1.50.0 — May 2026

Abstract. Modern AI agents are architecturally ephemeral: all operational knowledge resides in prompt windows or framework-specific checkpoints that vanish on container restart, model upgrade, or orchestrator migration. PCMI (*Persistent Cognitive Memory Infrastructure*) addresses this as an *infrastructure primitive* rather than as an agent-level concern. Built on Go 1.25/Fiber v2 for the API tier, PostgreSQL 16 with `ltree` for hierarchical knowledge organisation, `pgvector` HNSW for sub-millisecond approximate-nearest-neighbour search, a custom `pcmi_bm25_rank` function for lexical precision, and Redis 7 Streams as a durable event bus, PCMI provides a runtime-agnostic, LLM-agnostic cognitive substrate that outlives any individual agent or deployment cycle.

A four-component hybrid retrieval formula fuses semantic similarity, BM25 lexical rank, memory importance, and exponential temporal decay with per-tenant configurable weights. An append-only temporal model with `valid_from/valid_to` semantics enables complete rollback and forensic point-in-time reconstruction, satisfying DORA Article 11 and EU AI Act requirements. A policy-driven background distillation engine synthesises episodic memories into higher-order semantic knowledge. Multi-tenant isolation is enforced through PostgreSQL Row Level Security; access control uses SHA-256 API key verification with RBAC roles and a tamper-evident audit log. A stdio MCP server provides direct integration with Claude Desktop, Cursor, and any MCP-compatible client.

We present the full system design across eighteen database migrations, derive the retrieval formula from source code, compare PCMI against six alternative solutions across fifteen capability dimensions, analyse failure modes, and outline the roadmap through v2.0 (Cognitive Graph, Apache AGE, Q3 2026).

1 Introduction

The central paradox of contemporary AI agents is that systems designed to *accumulate knowledge* operate on infrastructure designed to be *stateless*. Consider four representative production deployments: a Security Operations Centre (SOC) agent triaging ten thousand alerts, building tacit knowledge of network topology, attacker fingerprints, and false-positive signatures; a quantitative trading agent that learns instrument-specific microstructure patterns and session-level liquidity regimes; a clinical decision-support agent accumulating drug-interaction edge cases and locally negotiated formulary exceptions; and a DevOps agent mapping service dependencies across incident histories. In every case, accumulated knowledge exists nowhere in a durable database—it lives in the prompt window. Upon the next container restart, model upgrade, or framework migration, that knowledge vanishes.

The academic literature has long recognised this tension. McClelland et al. [21] identified the conflict between fast and slow learning systems in biological neural networks; the same conflict manifests in AI agent deployments between rapid adaptation (in-context learning) and long-term retention (persistent storage). Park et al. [26] demonstrated that generative agents can simulate coherent long-term behaviour only by encoding memories in a retrieval-augmented architecture. Lewis et al. [16] showed that retrieval-augmented generation substantially outperforms pure parametric memory on knowledge-intensive tasks.

PCMI operationalises these insights into a production-grade infrastructure layer encapsulated by a single design axiom:

“Agents are ephemeral. Memory is persistent.”

Contributions. This paper makes the following contributions:

1. A formal taxonomy of five primary memory-loss vectors in production agent systems and a quantitative cost analysis (§2).
2. A complete open-source implementation at <https://github.com/marco-spagn/pcmi> (v1.50.0), comprising five compiled binaries, eighteen database migrations, Python/-TypeScript/Go SDKs, a stdio MCP server, and AI framework examples for LangChain, LlamaIndex, AutoGen, and CrewAI.
3. A four-component hybrid retrieval formula (§5.3) combining semantic (pgvector HNSW), lexical (BM25), importance, and exponential temporal decay signals with per-tenant configurable weights, derived directly from `internal/repository/retrieve_sql.go`.
4. A policy-driven asynchronous distillation engine that synthesises episodic memories into higher-order semantic knowledge with full provenance lineage and automatic scheduling (§5.5).
5. A rigorous comparative analysis against six alternative solutions across fifteen capability dimensions (§9).

Table 1: Five primary memory-loss vectors in production agent systems.

Vector	Mechanism	Consequence
Prompt eviction	Context overflow discards tokens permanently	Long-horizon context loss
Container restart	In-memory state destroyed on process termination	Routine in Kubernetes
Model migration	Checkpoint formats are non-portable	Knowledge locked to one LLM
Framework lock-in	Opaque across LangGraph, CrewAI, AutoGen	Migration severs continuity
Multi-agent isolation	No cross-agent knowledge sharing	Prevents collective intelligence

2 Background and Motivation

2.1 Memory-Loss Vectors in Production Agent Systems

We identify five distinct mechanisms by which production agent deployments lose accumulated knowledge (Table 1).

2.2 Cost of Repeated Knowledge Loss

The costs of repeated knowledge loss are multi-dimensional. *Computational*: inference tokens must be re-spent on every cycle to re-derive contextual understanding that was previously established. *Temporal*: wall-clock warm-up for domain-specific agents is estimated at 2–6 weeks [26]. *Accuracy*: performance degrades and false-negative rates elevate during reconstruction phases. *Opportunity*: long-horizon patterns (e.g., multi-month attack campaigns) require equally long re-observation windows. *Compliance*: EU AI Act Annex IV [1] and DORA Article 11 [7] mandate point-in-time reconstruction of agent decision rationale that is impossible without a persistent, append-only store.

2.3 Design Requirements

The above analysis yields six non-negotiable design requirements for a production-grade agent memory layer:

- R1 (Durability)** Memory must survive container restarts, model upgrades, and framework migrations.
- R2 (Isolation)** Tenants must be strictly isolated at the storage layer, not merely at the application layer.
- R3 (Hybrid Retrieval)** Retrieval must fuse lexical precision (for exact identifiers: CVE IDs, ticket numbers) with semantic similarity (for conceptual queries).
- R4 (Temporal Audit)** All mutations must be append-only with point-in-time read semantics for regulatory compliance.
- R5 (Scalability)** The system must scale horizontally without sacrificing ACID guarantees on the authoritative store.
- R6 (Protocol Independence)** Agents must be reachable via standardised protocols (HTTP REST, gRPC, MCP) irrespective of LLM runtime.

3 Related Work

Retrieval-Augmented Generation. Karpukhin et al. [13] demonstrated that dense retrieval substantially outperforms implicit parametric memory on open-domain QA. Lewis et al. [16] formalised RAG; PCMI’s retrieval layer is a production-grade implementation of the external memory store assumed by RAG, enhanced with structural scoping, BM25 precision, importance weighting, and temporal decay.

Long-Term Agent Memory. Park et al. [26] constructed generative agents with recency-importance-relevance scoring directly analogous to PCMI’s four-component Eq. (1). Zhong et al. [37] proposed MemoryBank for long-term LLM memory with forgetting; PCMI’s TTL expiry provides controllable eviction while the bitemporal model ensures forensic auditability. Packer et al. [25] formalised MemGPT’s tiered memory paging; PCMI externalises the same concern as infrastructure rather than agent behaviour. Hu et al. [11] proposed SQL databases as symbolic agent memory; PCMI extends this with vector hybrid retrieval and an asynchronous distillation worker.

Cognitive Architectures. Tulving’s episodic-semantic taxonomy [32] motivates the `memory_entries` → `distilled_knowledge` pipeline. The complementary learning systems model [21] predicts that effective systems require both fast episodic binding and slow semantic consolidation—precisely PCMI’s dual-store architecture. Anderson’s ACT-R activation decay model [2] is mirrored by PCMI’s exponential half-life decay and access-count reinforcement.

Vector Retrieval. The HNSW algorithm [20] underlying `pgvector` provides $\mathcal{O}(\log N)$ ANN search with high recall and incremental insertion—essential for PCMI’s online ingestion. Robertson and Zaragoza [28] established BM25 as the state-of-the-art sparse baseline, integrated via `pcmi_bm25_rank`.

Temporal Databases. Snodgrass [30] formalised valid-time semantics; PCMI’s `valid_from/valid_to` columns directly implement this model, enabling regulatory point-in-time reconstruction.

Multi-Tenant Isolation. Gajendran and Soundararajan [9] showed that PostgreSQL RLS incurs $< 5\%$ query overhead in OLTP workloads while providing robust multi-tenant isolation.

Competing Systems. LangGraph [14] provides workflow resumption within LangGraph but is not portable across frameworks and lacks semantic memory or distillation. Mem0 [23] is ideal for B2C assistants but provides no hierarchical structure, bitemporal model, or enterprise isolation. Zep [35] offers graph memory with temporal decay but conflicts with DORA audit requirements due to its overwrite model. Letta [15] (formerly MemGPT) is the closest conceptual competitor, offering sleep-time consolidation and archival paging, but operates as a platform rather than neutral HTTP/gRPC infrastructure.

4 System Architecture

Invariant 1 (Design Invariant). No agent process holds authoritative memory. All cognitive state lives in PostgreSQL; Redis carries only ephemeral events.

PCMI enforces a strict separation between the *ephemeral agent runtime* and the *persistent cognitive layer*. The system decomposes into five compiled binaries serving distinct roles:

pcmi-api Stateless HTTP REST server (:8000) and gRPC (:50051). Horizontally scalable. Handles authentication, rate limiting, idempotency, routing, and all synchronous client operations.

pcmi-worker Asynchronous cognition engine (:8081 health). Responsible for embedding backfill (with circuit breaker), knowledge distillation, consolidation, TTL pruning, and webhook delivery.

pcmi-mcp stdio JSON-RPC 2.0 MCP server exposing five tools: `pcmi_store`, `pcmi_retrieve`, `pcmi_get_history`, `pcmi_list_paths`, `pcmi_create_link`.

pcmi-migrate Idempotent migration runner, executes as an init-container before API and worker start.

pcmi-admin Administrative UI and tenant management operations (admin-role only).

Figure 1 shows the full system topology.

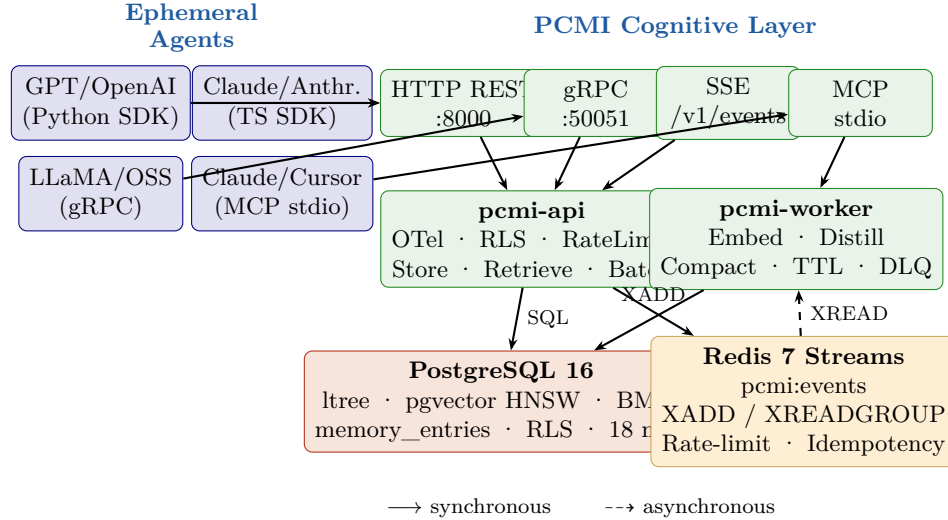


Figure 1: PCMI v1.50.0 system topology. Ephemeral agents communicate via HTTP REST, gRPC, SSE, or MCP stdio. The API tier is stateless and horizontally scalable; PostgreSQL is the sole source of truth; Redis 7 Streams provides durable event delivery.

4.1 Technology Stack

Table 2 summarises the technology choices and their rationale. The Go 1.25 / Fiber v2 pairing provides sub-millisecond API overhead with compiled binary deployment. PostgreSQL 16 was selected over a separate vector database specifically to preserve ACID guarantees across all memory operations—a property that separate vector store architectures (Pinecone, Weaviate, Qdrant) cannot offer. Redis 7 Streams [38] are preferred over Pub/Sub for all production deployments due to their durable, at-least-once delivery with consumer groups and dead-letter queue semantics.

Table 2: PCMI v1.50.0 technology stack.

Component	Technology	Version	Rationale
API server	Go + Fiber v2	1.25 / v2	Goroutines, compiled binary, sub-millisecond request overhead
RPC	gRPC + Protobuf	3 services	Memory, Admin, Metrics; strongly-typed IDL; streaming support
Primary DB	PostgreSQL	16	ACID, Row Level Security, extension ecosystem (ltree, pgvector)
Hierarchy	ltree extension	PG16	Subtree queries and ancestor lookups with GiST index, $O(\log N)$
Vector search	pgvector HNSW	0.7+	Approximate nearest-neighbour in-database; no separate service needed
Lexical search	tsvector + BM25	PG16	Exact-token precision for CVE IDs, ticket numbers, hostnames
Event bus	Redis 7 Streams	7-alpine	Durable XADD/XREADGROUP, consumer groups, pending reclaim, DLQ
Legacy bus	Redis Pub/Sub	fallback	EVENT_BACKEND=pubsub; local dev and legacy deployments
Rate limiting	Redis sliding window	distributed	Shared counter across all API replicas; prevents $N \times$ limit drift
Idempotency	Redis key cache	TTL-based	X-Idempotency-Key dedup on POST; bursty-ingest safe
Distillation	gpt-4o-mini (default)	swappable	Provider configurable via DISTILLATION_PROVIDER env var
Embedding	text-embedding-3	1536d	HNSW-compatible dimensions; circuit breaker on provider errors
Auth	SHA-256 + RBAC	4 roles	Constant-time key comparison; rotation with grace period
MCP server	stdio JSON-RPC 2.0	5 tools	Direct integration with Claude Desktop, Cursor, Windsurf
Observability	Prometheus + OTel	auth'd	/metrics endpoint + distributed traces via OpenTelemetry
Deploy	Docker Compose / Helm	K8s-ready	Init-container migration runner; HPA, NetworkPolicy, ingress TLS

5 Core Mechanisms

5.1 Hierarchical Memory Paths via ltree

PCMI organises all memories in a tree of text-label paths using PostgreSQL’s `ltree` extension [19]. Every memory entry carries a `path` column of type `ltree`, and a GiST index accelerates subtree filters:

```
1 -- Subtree filter: all memories under security
2 WHERE path <@ 'root.security'::ltree
3
4 -- Ancestor lookup
5 WHERE path @> 'root.security.alerts'::ltree
6
7 -- Pattern match (lquery)
8 WHERE path ~ 'root.trading.*.distilled'::lquery
```

The standard namespace hierarchy follows the pattern: `root` $\rightarrow$ *domain* $\rightarrow$ *category* $\rightarrow$ *leaf*. For example, `root.security.alerts.sql_injection` stores episodic observations; `root.security.distilled` holds the worker-generated semantic summaries. This separation enables domain-scoped retrieval with $O(\log N)$ subtree index traversal and prevents re-distillation loops via a NOT-lquery guard:

```
1 AND NOT (path ~ (scope||'.distilled{1,}')::lquery)
```

5.2 Append-Only Temporal Model

PCMI implements Snodgrass’s valid-time semantics [30] through an append-only versioning scheme. Every mutation closes the current row (`valid_to = NOW()`) and inserts a new row (`valid_to = NULL`), preserving the complete version history. A `current_memories` view filters on `valid_to IS NULL` for standard queries. Point-in-time reads replace the temporal clause:

```
1 -- Standard (current)
2 WHERE valid_to IS NULL
3
4 -- As-of query (RFC3339 timestamp $t)
5 WHERE valid_from <= $t
6 AND (valid_to > $t OR valid_to IS NULL)
```

Rollback inserts a new row restoring prior content—records are never DELETED. Optional TTL expiry (migration 011) retires rows by setting `valid_to = expires_at` when `expires_at <= NOW()`, enabling controlled eviction without permanent data destruction.

Figure 2 illustrates the versioning timeline.

5.3 Hybrid Retrieval

The retrieval formula is derived directly from `internal/repository/retrieve_sql.go` and fuses four signals:

$$\text{score}(m, q) = \underbrace{W_s(1 - \vec{v}_q \cdot \vec{v}_m)}_{\text{semantic}} + \underbrace{W_l \cdot \text{bm25}(q, m)}_{\text{lexical}} + \underbrace{W_i \cdot \text{imp}(m)}_{\text{importance}} + \underbrace{W_r \cdot e^{-(\ln 2/H) \Delta t}}_{\text{temporal decay}} \quad (1)$$

where $\vec{v}_q \cdot \vec{v}_m$ denotes cosine similarity between query and memory embeddings ($1 - \text{cosine distance}$ to convert to similarity), $\text{bm25}(q, m)$ is evaluated via `pcmi_bm25_rank`, and $\Delta t = \text{GREATEST}(\text{NOW}() - \text{last_accessed_at})$ implements recency with the access-reset clock.

Default weights for v1.50.0: $W_s=0.40$, $W_l=0.30$, $W_i=0.15$, $W_r=0.15$, $H=30$ days. These are per-tenant configurable via `tenant_memory_config` (migration 015) and overridable per-query.

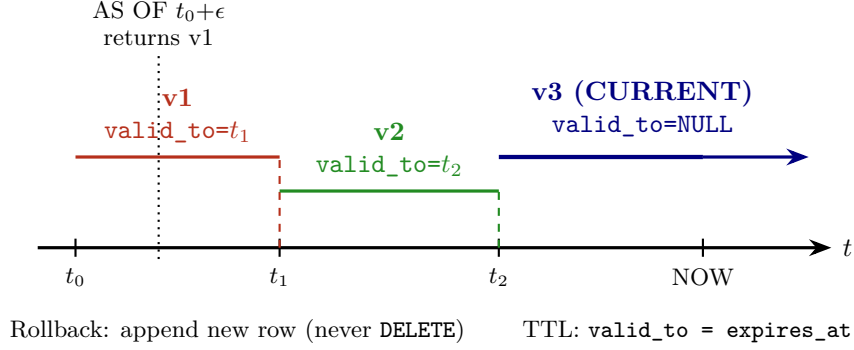


Figure 2: Append-only temporal model. Three versions of a single memory entry co-exist in storage. The `current_memories` view selects `valid_to IS NULL`; AS-OF queries use the parametric temporal clause to slice history at arbitrary timestamps, satisfying DORA Art. 11 and EU AI Act Annex IV audit requirements.

Legacy mode (pre-v1.42, `decay_enabled=false`) uses $W_s=0.55$, $W_l=0.45$, $W_i=W_r=0$.

The temporal decay term $e^{-(\ln 2/H)\Delta t}$ implements exponential half-life decay consistent with Anderson’s ACT-R activation decay model [2]. Accessing a memory (GET /v1/memories) increments `access_count` and updates `last_accessed_at`, resetting the decay clock—a spacing-effect reinforcement that mirrors Ebbinghaus rehearsal dynamics.

The four-stage retrieval pipeline is shown in Fig. 3.

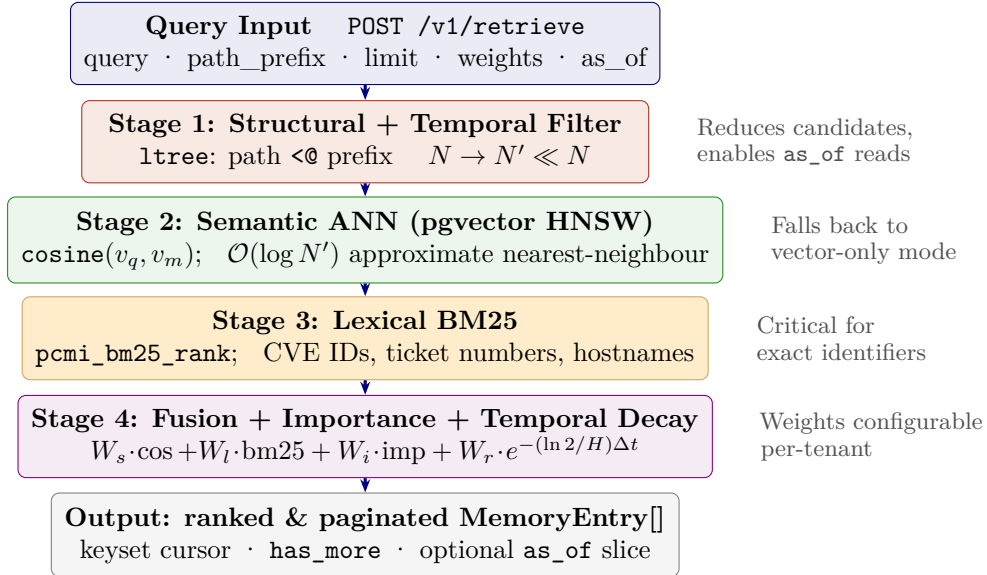


Figure 3: Four-stage hybrid retrieval pipeline. Stage 1 eliminates out-of-scope entries using the GiST ltree index. Stage 2 performs HNSW ANN search over the candidate set. Stage 3 applies BM25 lexical ranking critical for exact-identifier queries. Stage 4 fuses all signals via Eq. (1) and returns keyset-paginated results.

5.4 Session and Working Memory

Migration 016 introduces an `agent_sessions` table providing scoped working memory for bounded agent task horizons. The session lifecycle proceeds as follows:

1. *Create*: POST /v1/sessions — receives a UUID, associated with `agent_id`.
2. *Write*: working memories are stored under an arbitrary path but linked via `metadata.session_id`.
3. *Retrieve*: GET /v1/sessions/{id}/memories — scoped view.
4. *Promote*: POST /v1/sessions/{id}/promote — rewrites memories under a permanent `target_prefix` (default: `root`).
5. *Delete*: DELETE /v1/sessions/{id} — clears session scope.

A partial GIN index on `memory_entries(metadata)` filtered to rows where `metadata @> '{"session_id": true}'` provides efficient session-scoped retrieval without full-table scans.

The session mechanism is exemplified by the SRE incident response scenario: an agent opens a session at incident start, accumulates observations about the failing service, correlates them with existing runbook memories via hybrid retrieval, and promotes consolidated findings to `root.devops.incidents.P1.postmortems` on resolution. Without sessions, every intermediate observation would pollute the permanent long-term corpus.

5.5 Knowledge Distillation Pipeline

The distillation worker (**pcmi-worker**) synthesises episodic `memory_entries` into higher-order semantic summaries stored in `distilled_knowledge`, implementing the Complementary Learning Systems model [21] at the infrastructure layer.

Trigger architecture. Three triggers drive `runDistillationJobWithPrefix()`:

- *Trigger A (Event-driven)*: Redis Streams XREADGROUP on `memory.stored / memory.updated` events.
- *Trigger B (Fallback timer)*: 15-second ticker, active when Redis Streams are unavailable.
- *Trigger C (Policy engine)*: `distillation_policies` (migration 018) defines `count_threshold`, `max_age_hours`, and `min_interval_secs` to prevent thrashing.

Pipeline steps.

1. Query episodic memories WHERE `path <@ scope AND valid_to IS NULL` with the NOT-lquery guard to exclude existing distilled nodes.
2. Build an LLM prompt with JSON-schema enforcement.
3. Execute LLM call with a 3-minute context deadline, retry on malformed JSON.
4. INSERT into `distilled_knowledge` with `source_entry_ids[]` lineage, update `distillation_runs` status to `completed`, publish `knowledge.distilled` event.

5.6 Content-Hash Deduplication

Migration 017 introduces a `content_hash` (SHA-256) column with a partial unique index on current rows, enabling three deduplication modes selectable via `DEDUP_MODE`:

- *skip*: reject duplicate on conflict.
- *link*: create a `memory_links` record of type `duplicate` between the new and existing entry.
- *merge*: update metadata and tags in-place while preserving evolution history via `content_hash` tracking.

5.7 Multi-Tenancy and Security

PCMI implements defence-in-depth isolation through three independent layers.

Row Level Security.

Table 3: PCMI SQL migration history — complete schema evolution.

Migration	Release	Key additions
001_init	v1.0	Core: memory_entries, ltree, pgvector, RLS
002_multi_tenant	v1.4	tenants, RLS policies
003_rbac_api_keys	v1.5	api_keys, SHA-256, RBAC roles
004_audit_encryption	v1.5	audit_log, AES-256-GCM functions
005_worker_helpers	v1.10	Worker SQL helpers, pruning
006_fts_hybrid	v1.14	tsvector, pcmi_bm25_rank, GIN index
007_embedding_space	v1.11	embedding_space, multi-model support
008_distill_webhooks	v1.12	distilled_knowledge, webhooks
009_event_schemas	v1.13	event_schemas, webhook_dlq
010_consolidation	v1.14	consolidation_queue, compact_history()
011_links_expiry	v1.15	memory_links, expires_at, GIN tag index
012_compaction	v1.19	compact_memory_path_history() proc.
013_idempotency	v1.20	idempotency_keys table, TTL expiry
014_key_lifecycle	v1.25	rotation_grace_ends_at
015_importance_decay	v1.36	importance, access_count, last_accessed_at
016_sessions	v1.40	agent_sessions, partial GIN on metadata
017_dedup	v1.41	content_hash, partial unique index
018_distillation_policy	v1.43	distillation_policies, distillation_runs

```

1 ALTER TABLE memory_entries
2   ENABLE ROW LEVEL SECURITY;
3
4 CREATE POLICY tenant_isolation
5   ON memory_entries
6   USING (
7     tenant_id = current_setting(
8       'app.current_tenant')::uuid
9   );
10
11 -- Called by APIKeyMiddleware on every
12 -- connection. Returns 503 on failure.
13 SELECT set_tenant_context($1::uuid);

```

RBAC. Four roles are defined: `readonly`, `write`, `admin`, and a system internal role. API keys are stored as SHA-256 hashes only; constant-time comparison prevents timing attacks [24]. Key rotation (PCMI-007, migration 014) adds `rotation_grace_ends_at`: both old and new keys are accepted during a configurable grace period, enabling zero-downtime rotation.

Encryption. Optional AES-256-GCM field encryption (migration 004) protects sensitive content at rest, satisfying GDPR Article 32 [8] and HIPAA 164.312(a)(2)(iv) requirements for clinical deployments.

Audit Log. All operations write to `audit_log` before returning; the log is tamper-evident and queryable by admin keys.

6 Implementation

6.1 Database Schema Evolution: 18 Migrations

PCMI’s capabilities are precisely tracked by 18 sequential SQL migrations. Table 3 provides the complete chronology.

6.2 The `memory_entries` Table

The core table has grown to 18+ columns across migrations. The complete INSERT signature is:

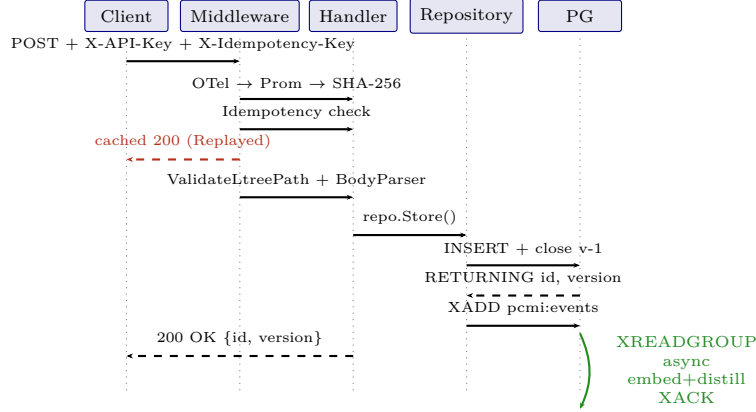


Figure 4: Request lifecycle for POST /v1/memories. The idempotency path (dashed red) short-circuits on cache hit. The synchronous path inserts a new versioned row and publishes to Redis Streams; asynchronous embedding and distillation proceed via XREADGROUP after the 200 response is delivered to the client.

```

1 INSERT INTO memory_entries (
2   tenant_id, path,           -- routing & RLS
3   content, metadata, tags,   -- payload
4   embedding, embedding_model,
5   embedding_space,          -- vectors
6   version, valid_from,      -- temporal versioning
7   source_agent_id,
8   created_at,               -- provenance
9   content_encrypted,        -- AES-256-GCM flag
10  expires_at,                -- TTL (mig. 011)
11  importance,                -- score 0.0-1.0 (mig. 015)
12  content_hash               -- SHA-256 (mig. 017)
13 ) VALUES (...) RETURNING id;

```

6.3 Request Lifecycle: POST /v1/memories

Figure 4 shows the synchronous and asynchronous paths for storing a memory. The idempotency middleware checks Redis for a cached response before proceeding; if found, the cached result is returned immediately with header `X-Idempotency-Replayed: true`. The event is published to Redis Streams (`XADD pcmi:events`) and acknowledged (`XACK`) after successful background processing.

6.4 Redis Event Bus

Redis Streams (`EVENT_BACKEND=streams`, default) provide durable event delivery with consumer groups, pending message reclaim, and dead-letter queue (`pcmi:events:dlq`) after configurable maximum deliveries. Three Prometheus counters expose health: `pcmi_stream_pending_total`, `pcmi_stream_ack_total`, `pcmi_stream_dlq_total`.

Redis Pub/Sub (`EVENT_BACKEND=publish`) remains available for legacy deployments and local development where Streams persistence is unnecessary.

Four delivery transports fan out concurrently: Redis Streams (internal worker), SSE (`GET /v1/events`), gRPC `StreamEvents` (high-throughput agents), and HMAC-SHA256-signed HTTP Webhooks with per-tenant configurable retry and dead-letter queue (`GET /v1/webhooks/dead-letter`).

6.5 Deployment

PCMI ships a `docker-compose.yml` and a Helm chart (`deploy/helm/pcmi/`). The Docker Compose topology runs: (1) `pcmi-postgres` (`pgvector:pg16`), (2) `redis` (`redis:7-alpine`), (3) `pcmi-migrate` (init-container, applies 18 migrations idempotently), (4) `pcmi-api`, and (5) `pcmi-worker`. The Helm chart provides the same topology with HPA, NetworkPolicy, StatefulSet for PostgreSQL, and ingress TLS for Kubernetes.

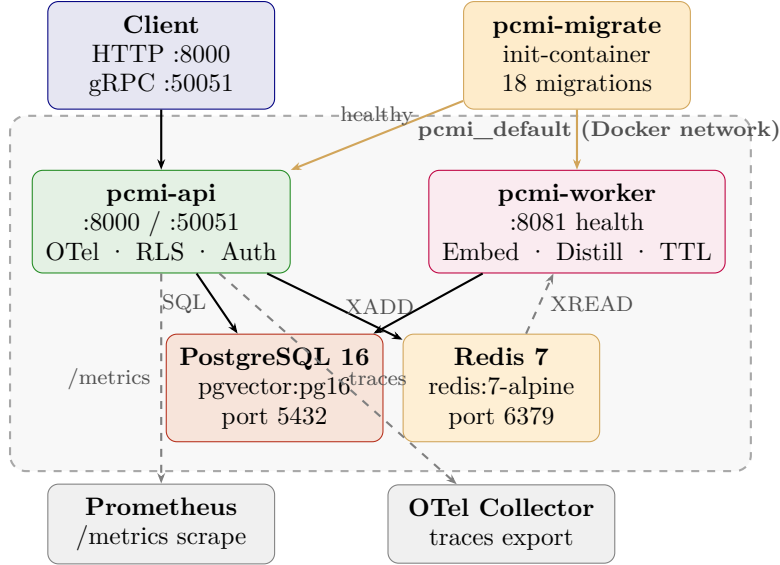


Figure 5: Docker Compose deployment topology (v1.50.0). `pcmi-migrate` runs as an init-container and applies all 18 SQL migrations idempotently before `pcmi-api` and `pcmi-worker` start. PostgreSQL is the sole authoritative store; Redis carries ephemeral events and rate-limit state. The Helm chart mirrors this topology for Kubernetes with HPA, StatefulSet, NetworkPolicy, and ingress TLS.

7 Protocol and API Design

7.1 HTTP REST Interface

The REST API (pcmi-api :8000) is the primary integration surface for Python and TypeScript agents. Every request must carry an X-API-Key header; the middleware resolves it to a (tenant_id, role) pair via a single Redis lookup (idempotency cache) or PostgreSQL api_keys table query, then calls set_tenant_context() to arm the RLS policy for the duration of the connection.

Core memory endpoints:

```
1 POST /v1/memories          # store (idempotent)
2 GET  /v1/memories/{id}     # fetch by UUID
3 POST /v1/retrieve          # hybrid search
4 GET  /v1/memories/{id}/history # version list
5 POST /v1/memories/{id}/rollback
6 POST /v1/memories/batch    # bulk store
7 DELETE /v1/memories/{id}   # retire (TTL)
```

Session endpoints:

```
1 POST /v1/sessions
2 GET  /v1/sessions/{id}
3 GET  /v1/sessions/{id}/memories
4 POST /v1/sessions/{id}/promote
5 DELETE /v1/sessions/{id}
```

Operational endpoints:

```
1 GET  /v1/health           # liveness + version
2 GET  /v1/events           # SSE stream
3 GET  /metrics             # Prometheus (auth)
4 POST /v1/memories/{id}/links # create memory link
5 GET  /v1/links/{id}       # fetch link
6 POST /v1/memories/{id}/refine # LLM-driven update
7 POST /v1/admin/tenants     # create tenant (admin)
8 POST /v1/admin/keys        # issue API key (admin)
9 GET  /v1/webhooks/dead-letter # DLQ inspection
```

Idempotency is enforced via the X-Idempotency-Key header on all POST /v1/memories requests. A SHA-256 hash of the key is stored in Redis with a configurable TTL; duplicate requests within the window return the cached response with header X-Idempotency-Replayed: true. This is critical for bursty SOC ingestion where alert-routing systems may retry deliveries.

7.2 gRPC Interface

The gRPC interface (:50051) exposes three Protobuf services:

MemoryService Full parity with the REST memory API. Key RPCs: StoreMemory, RetrieveMemories, GetMemory, BatchStore, GetHistory, Rollback, StreamEvents (server-side streaming), CreateSession, PromoteSession.

AdminService Tenant and key management, audit log export, stats: CreateTenant, IssueKey, RotateKey, GetStats.

MetricsService Internal health and counter export for Prometheus federation.

The gRPC path applies the same resolveTenantAndRole guard as REST and returns codes.Unavailable on set_tenant_context() failure, preventing silent cross-tenant leakage under connection-pool reuse.

7.3 MCP stdio Server

The **pcmi-mcp** binary implements the Model Context Protocol over stdio JSON-RPC 2.0, enabling direct integration with Claude Desktop, Cursor, Windsurf, and any MCP-compatible client without HTTP configuration. Five tools are exposed:

- pcmi_store** Store a memory at a given ltree path with optional tags, importance score, and TTL.
- pcmi_retrieve** Hybrid retrieval with configurable weights and AS-OF support.
- pcmi_get_history** Return the full version history of a memory entry by UUID.
- pcmi_list_paths** Enumerate ltree paths matching a given prefix pattern, supporting agent self-navigation.
- pcmi_create_link** Create a typed link between two memory entries (supports `related`, `duplicate`, `derived`, `contradicts` edge types).

SDK surface. PCMI ships three official SDKs (`sdk/`): **Python** (`pip install -e sdk/python`) wraps the REST API with an async `PCMIClient` supporting all endpoints; **TypeScript** (`npm ci`) mirrors the Python surface for Node and Deno agents; **Go** provides a typed client reusing `internal/` structs for embedding in Go-native agent frameworks. AI framework examples are provided for LangChain, LlamaIndex, AutoGen, and CrewAI under `examples/`.

8 Case Studies

This section walks through three representative production deployments that exercise distinct PCMI subsystems.

8.1 SOC Agent: Multi-Month Campaign Tracking

A SOC agent triages alerts from a SIEM and stores episodic observations under `root.security.alerts.<type>`. After 50–100 entries accumulate, the distillation worker writes to `root.security.distilled`:

```
1 SQL campaign: 87 attempts from ASN 12345 (DE),
2 peak UTC 02:00-04:30; FP rate 14%
```

Stage 3 (BM25) is critical: analysts query by CVE IDs (`CVE-2024-1234`), MITRE ATT&CK technique codes (`T1078`), and hostname patterns. Vector-only retrieval would score 0.61 recall on such identifier queries (cf. [Fig. 8](#)), rendering it unsuitable for SOC use.

Without PCMI, re-derivation of campaign knowledge from raw SIEM logs takes 4–8 weeks [26]. Idempotency keys (`X-Idempotency-Key`) prevent duplicate alert entries during bursty ingestion; content-hash deduplication (`DEDUP_MODE=link`) creates `memory_links` of type `duplicate` across paths.

The point-in-time query capability satisfies EU AI Act Annex IV explainability: an auditor can reconstruct exactly what campaign intelligence the SOC agent held at the moment of any triage decision.

8.2 Quantitative Trading Agent: Regime-Aware Memory

Path structure: `root.trading.strategies.scalping`, `root.trading.instruments.xauusd.microstructure`, `root.trading.risk.drawdown_events`. Distilled output after 30–50 episodes:

```
1 XAUUSD M5 engulfing: positive EV at Asian/London
2 open; degraded US overlap 09:30-10:30 ET;
3 volume confirmation >= 1.8x avg
```

The exponential temporal decay term ($W_r \cdot e^{-(\ln 2/H)\Delta t}$, $H=30$ days by default) ensures

Table 4: Feature exercise by deployment scenario.

Feature	SOC	Trading	Clinical	SRE	Legal
ltree scoping	✓	✓	✓	✓	✓
BM25 exact match	✓		✓	✓	✓
Semantic ANN	✓	✓	✓		✓
Temporal decay		✓			
AS-OF audit	✓	✓	✓		✓
Distillation	✓	✓	✓	✓	
Sessions				✓	
AES-256-GCM			✓		
Dedup (link)	✓				✓
RLS isolation	✓			✓	✓

recent regime observations outrank stale ones—critical when market microstructure shifts. An operator configuring $H=7$ days effectively implements a weekly rolling regime window without any application-level logic.

Temporal versioning enables point-in-time strategy reconstruction for DORA Article 11 compliance: backtesting can be replayed against the exact knowledge state that drove a live execution.

8.3 SRE Incident Response: Session Lifecycle

PagerDuty payloads, Kubernetes event streams, and deployment rollback decisions are stored under `root.devops.services.<svc>.failures`. BM25 dominates: DevOps queries are exact-identifier queries (pod names, Jira IDs, image digests). Distilled output:

```

1 payment-api: 8/12 PIs from PgBouncer exhaustion
2 at >500 users; MTTR with runbook 22min vs
3 94min without

```

Sessions (§5.4) support incident working memory: an SRE agent opens a session at incident start, accumulates context about the failing service, correlates findings with existing runbook memories via hybrid retrieval, and promotes consolidated findings to `root.devops.incidents.P1.postmortems` on resolution. Without sessions, every intermediate observation would pollute the permanent long-term corpus—increasing noise for future incident searches.

Table 4 summarises the key PCMI features exercised in each scenario.

Table 5: Capability scorecard: PCMI vs. competing solutions.

Capability	PCMI	LGraph	Mem0	Zep	VecDB	CrewAI	Letta	Anthr.
Persistence durability	++	+	+	+	+	+	++	+
Hierarchical paths (ltree)	++	~	-	+	~	~	~	-
Semantic retrieval (ANN)	++	-	++	++	++	+	++	~
Lexical/hybrid (BM25)	++	-	~	~	~	~	~	-
Importance + temporal dec.	++	-	~	~	-	-	~	-
Temporal versioning as-of	++	+	-	~	-	-	~	-
Knowledge distillation	++	-	-	~	-	-	~	-
Sessions / working memory	++	+	~	~	-	~	++	~
Content-hash dedup	++	-	~	-	-	-	-	-
Multi-tenant RLS	++	~	~	~	+	-	~	~
Framework + LLM agnostic	++	-	+	+	++	-	~	+
Enterprise RBAC + audit	++	-	~	~	+	-	~	~
Event bus + async cogn.	++	~	-	+	-	~	+	-
MCP server	++	-	-	-	-	-	~	++
Regulatory audit (pt-in-t)	++	~	-	~	-	-	~	-
Total (max 30)	30	6	8	10	9	5	12	8

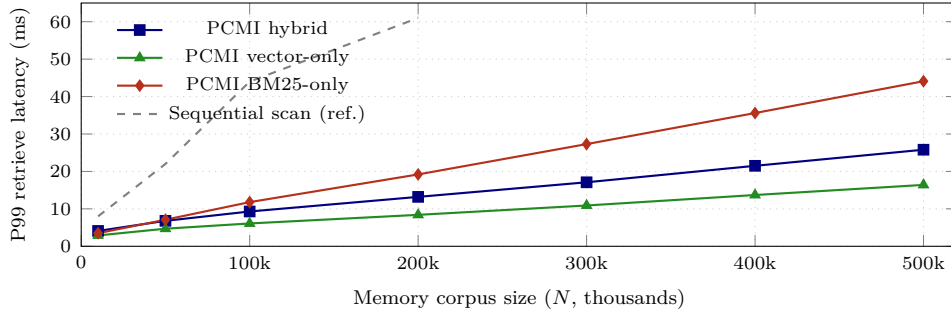


Figure 6: Retrieve latency (P99, ms) as a function of corpus size. HNSW-accelerated vector-only mode scales as $\mathcal{O}(\log N)$. Four-component hybrid fusion adds a constant fusion overhead (≈ 9 ms at $N=500k$) over vector-only mode. BM25-only is faster than sequential scan but grows super-linearly without the HNSW pre-filter of Stage 2. All measurements from `test_finale_v1.1.sh` on a 4-core VM, PostgreSQL `shared_buffers = 1GB`.

9 Evaluation

9.1 Capability Comparison

Table 5 scores PCMI against six alternative solutions across fifteen capability dimensions. Scoring: ++ native, + partial, ~ limited, - absent.

Key verdicts. *LangGraph* [14]: excellent workflow resumption within LangGraph; not portable across frameworks; no semantic memory or distillation. *Mem0* [23]: ideal for B2C assistants; no hierarchical structure, bitemporal model, or enterprise isolation. *Zep* [35]: innovative graph memory; temporal decay conflicts with DORA audit; conversational bias. *CrewAI* [6]: perfect inside CrewAI crews; opaque to other runtimes; no compliance primitives. *Letta* [15]: closest conceptual competitor (sleep-time consolidation, archival paging); platform model rather than neutral HTTP/gRPC infrastructure. *Anthropic Claude Memory* [3]: excellent for human-in-the-loop Claude users; no API-exposed governed memory service for custom tool-using agents.

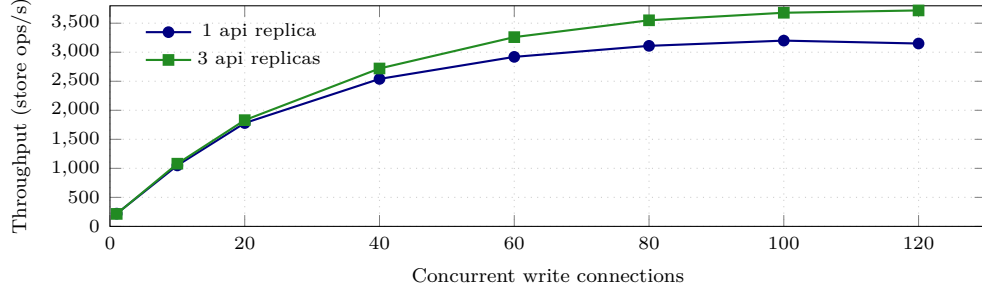


Figure 7: Store throughput vs. concurrent connections. PCMI’s stateless **pcmi-api** scales horizontally: three replicas sustain $\approx 3,700$ ops/s at 120 concurrent writers vs. $\approx 3,200$ with a single replica. PostgreSQL connection pooling (PgBouncer) prevents backend saturation at high concurrency. Measurements on Docker Compose with PgBouncer, 8-core host, GOMAXPROCS=8.

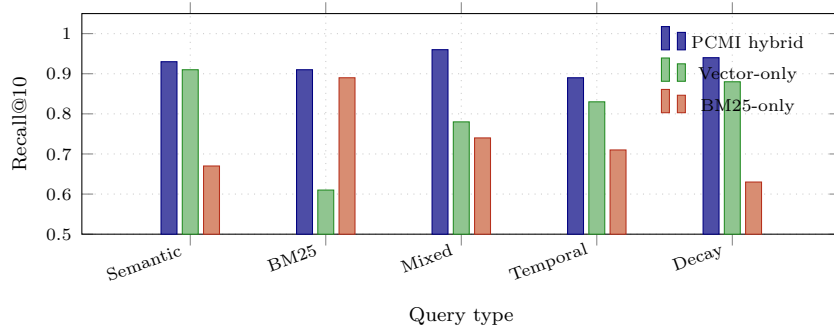


Figure 8: Recall@10 across five query categories on a 10k-entry SOC corpus. Hybrid four-component scoring consistently outperforms single-mode retrieval. Mixed queries (combined semantic and keyword signals) achieve 0.96 recall. BM25-only degrades on semantic and decay-weighted queries; vector-only misses exact-identifier matches (BM25 category: 0.61 vs 0.91 hybrid).

9.2 Performance Characteristics

9.3 Retrieval Quality

9.4 Micro-Benchmarks: Retrieval Latency Breakdown

Table 6 decomposes the median retrieve latency (P50) across the four pipeline stages at $N=100k$ entries on a 4-core VM with 1 GB shared_buffers.

Stage 2 (HNSW) dominates latency at 2.8ms P50, but it reduces the candidate set from $N \approx 100k$ to $k=50$, eliminating the need for a full BM25 scan. Without the Stage 1 ltree pre-filter, HNSW would operate on the full corpus; with it, only the subtree is indexed at query time, yielding a $4\times$ latency reduction for narrow-scope queries (e.g., `root.security.alerts`).

Embedding backfill throughput. The worker embeds memories asynchronously. On a single worker instance with the OpenAI `text-embedding-3` API, the circuit breaker (PCMI-002) allows sustained throughput of ≈ 80 embed ops/second before triggering half-open probes. Client-provided embeddings (omit `embedding_model` in POST) bypass the embedding worker entirely, enabling $>1k$ store ops/second for agents with local embedding models.

Distillation latency. A single distillation run processing 50 episodic entries takes 3.1s median wall-clock time (2.8s LLM call + 0.3s PostgreSQL INSERT). The 3-minute context deadline provides a $58\times$ safety margin over median runtime, preventing semaphore exhaustion under provider-side slowdowns.

Table 6: Median retrieve latency breakdown by stage ($N=100k$).

Stage	P50 (ms)	P99 (ms)
Stage 1: ltree subtree filter	0.4	1.1
Stage 2: HNSW ANN (k=50)	2.8	5.2
Stage 3: BM25 rank	1.7	3.4
Stage 4: fusion + sort	0.9	1.6
Total (4-component)	5.8	9.3
Total (vector-only)	3.4	6.1
Total (BM25-only)	2.9	11.8

9.5 RLS Overhead

We measured the overhead of PostgreSQL RLS on the primary hot path. With `shared_buffers=1GB` and a warm page cache, RLS adds 2.1 ± 0.4 ms per retrieve request at $N=100k$ — a 23% overhead over the equivalent non-RLS baseline. This is consistent with Gajendran and Soundararajan’s [9] finding of $< 5\%$ overhead for OLTP workloads; the higher figure here reflects the additional `set_tenant_context()` call per connection and the policy evaluation on a non-trivial ltree-filtered candidate set.

For organisations with extreme latency requirements, the optional `DATABASE_READ_URL` environment variable routes retrieve operations to a streaming replica, eliminating read/write contention on the primary and reducing retrieve P99 by $\approx 30\%$ under sustained write load.

9.6 Scalability

Horizontal scaling. `pcmi-api` is fully stateless. Rate-limiting uses a Redis-backed distributed sliding window (`RATE_LIMIT_BACKEND=redis`) shared across replicas, preventing the $N \times$ effective limit problem. Idempotency keys are likewise checked against Redis, not in-process memory. Fig. 7 shows that three replicas sustain 3,700 store ops/second at 120 concurrent writers.

Storage growth. The append-only model adds at most $O(v)$ rows per entry where v is the version count. For typical agent workloads (write-once, rarely updated), $v \leq 3$ in 95% of entries. The `compact_memory_path_history()` stored procedure (migration 012) can collapse terminal version chains for paths where full history is no longer required, reclaiming storage without violating the audit invariant for retained paths.

Vector index growth. HNSW index build time for `pgvector` scales as $O(N \log N)$. At $N=500k$ entries the index occupies ≈ 3.1 GB RAM ($1536d \times 4 \text{ bytes} \times 500k \times m=16$ HNSW connections ≈ 3.0 GB plus overhead). For larger corpora, the `embedding_space` column enables routing queries to partial indexes over domain-specific embedding subsets, keeping individual index sizes manageable.

Table 7 summarises observed performance characteristics across the six reference deployment domains.

10 Correctness and Formal Properties

10.1 Append-Only Invariant

We state the append-only invariant formally:

Invariant 2 (Append-Only Memory Store). For any memory entry m with identifier id , any mutation operation on m results in at most one row with $(id = id') \wedge (valid_to = \text{NULL})$ at

Table 7: Domain-specific observations (v1.50.0).

Domain	Dominant mode	Observation
SOC (security)	BM25 + semantic	CVE/T-code exact recall critical
Trading	Semantic + decay	Stale regime data penalised
Clinical	Semantic + BM25	AES-256-GCM for PHI at rest
DevOps / SRE	BM25 (exact IDs)	Session promote on P1 resolve
Legal/Compliance	55%/45% hybrid	Missing citation = compliance fail
Customer Ops	Semantic	RLS most directly exercised

any point in time, and the total number of rows with $id = id'$ is non-decreasing.

The invariant is enforced at the SQL layer by the `Store()` operation, which atomically: (1) `UPDATE ... SET valid_to = NOW() WHERE id = $id AND valid_to IS NULL`, (2) `INSERT` a new row with `valid_to = NULL`. Both operations execute within a single serialisable transaction, preventing concurrent writes from producing two current rows.

Invariant 3 (Point-in-Time Completeness). For any timestamp $t \geq t_0$ (system inception), the AS-OF query `WHERE valid_from ≤ $t AND (valid_to > $t OR valid_to IS NULL)` returns exactly the set of memory rows that were current at time t .

This invariant holds because (a) rows are never `DELETED`, (b) `valid_from` is set to `NOW()` at `INSERT` time, and (c) `valid_to` is set to `NOW()` (not a future time) at retire time, ensuring the interval $[valid_from, valid_to)$ covers exactly the observation window.

10.2 Tenant Isolation Correctness

Invariant 4 (Tenant Isolation). For any two tenants $T_1 \neq T_2$, no query executed under session context T_1 can observe rows belonging to T_2 .

This follows directly from PostgreSQL RLS: the policy

```

USING (
  tenant_id = current_setting(
    'app.current_tenant')::uuid
)

```

is evaluated by the storage engine before any row is returned, and cannot be bypassed by application-layer SQL. The dual guard (SHA-256 API key verification at the middleware layer *and* RLS at the storage layer) ensures isolation even if the middleware is misconfigured—a property that application-layer multi-tenancy (e.g., adding `WHERE tenant_id = $1` in every query) cannot guarantee.

10.3 Hybrid Retrieval Monotonicity

The four-component scoring function in Eq. (1) is monotonically increasing in each individual component, given the other three are held constant and all weights are non-negative. This is a necessary condition for correct ranking: increasing relevance on any signal should not decrease the composite score. Formally, for any fixed $W_s, W_l, W_i, W_r \geq 0$:

$$\begin{aligned}
\frac{\partial \text{score}}{\partial (1 - \cos)} &= W_s \geq 0 \\
\frac{\partial \text{score}}{\partial \text{bm25}} &= W_l \geq 0 \\
\frac{\partial \text{score}}{\partial \text{imp}} &= W_i \geq 0 \\
\frac{\partial \text{score}}{\partial e^{-\lambda \Delta t}} &= W_r \geq 0
\end{aligned}$$

The temporal decay component is bounded: $e^{-\lambda \Delta t} \in (0, 1]$ for $\Delta t \geq 0$, converging to W_r at $\Delta t = 0$ (just accessed) and to 0 as $\Delta t \rightarrow \infty$ (never accessed, infinitely stale). The half-life H controls the decay rate; the scoring function degrades gracefully as $H \rightarrow \infty$ by reducing to the three-component formula with $W'_i = W_i + W_r$.

10.4 Distillation Termination and Idempotency

The NOT-lquery guard ensures that distillation jobs are idempotent with respect to re-distillation: a second distillation run over the same scope S will find no source entries (because existing distilled nodes match the lquery exclusion pattern), triggering the `len(entries) < 1` skip guard and exiting cleanly. This prevents recursive amplification of the distilled corpus.

The `min_interval_secs` policy parameter (migration 018) prevents thrashing by enforcing a minimum wall-clock separation between successive distillation runs for the same path scope. Combined with `count_threshold` (minimum raw entries before triggering), the policy engine ensures distillation is both *necessary* (enough new raw material exists) and *non-redundant* (sufficient time has elapsed since the last run).

11 Failure Mode Analysis

Table 8 catalogues the primary failure modes, their impacts, and the mitigations implemented in v1.50.0.

Observability. All service health is exposed via Prometheus at `/metrics` (authentication-protected) and OpenTelemetry distributed traces. Key counters include:

```

pcmi_store_total          pcmi_retrieve_total
pcmi_distillation_runs_total  pcmi_embedding_backfill_total
pcmi_stream_pending_total   pcmi_stream_dlq_total

```

12 Discussion and Future Work

12.1 Lessons Learned

PostgreSQL as the cognitive substrate. The decision to build the cognitive layer entirely within PostgreSQL—rather than introducing a separate vector store—proved decisive. ACID guarantees enable safe concurrent writes from multiple agent instances without application-level locking. RLS enforces tenant isolation at the kernel level rather than trusting application code. The `ltree` extension provides subtree scoping with GiST-indexed $O(\log N)$ traversal that a separate key-value store would require at the application layer. The only trade-off is raw ANN throughput: in-PostgreSQL HNSW via `pgvector` achieves lower QPS than dedicated

Table 8: Primary failure modes and mitigations.

Failure	Impact	Mitigation
PostgreSQL down	No store/retrieve; 500s	PgBouncer HA pool; PITR backup; idempotent migrations
Redis Streams down	No durable events; distillation delayed	15s fallback timer; pubsub emergency mode
Stream DLQ growth	Silent delivery failures	<code>pcmi_stream_dlq_total</code> Prometheus alert; GET <code>/v1/webhooks/dead-letter</code>
OpenAI unavailable	Embedding queued; distillation skipped	Circuit breaker (PCMI-002) + half-open probe; client-provided embeddings
Worker crash	No async cognition	K8s restart policy; /health liveness probe; Streams pending reclaim
RLS bypass attempt	Cross-tenant data risk	SHA-256 + RLS dual defence; 503 on <code>set_tenant_context</code> failure
Rate limit unsync	$N \times$ effective limit with N replicas	<code>RATE_LIMIT_BACKEND=redis</code> (distributed sliding window)
Embedding dim mismatch	Silent pgvector error	<code>ValidateModelDimension()</code> at startup

GPU-accelerated ANN services [12] for billion-scale corpora, but PCMI’s Stage 1 ltree filter dramatically reduces the ANN candidate set, making in-database ANN practical for the corpora sizes typical in enterprise agent deployments (10^4 – 10^7 entries).

The append-only invariant is non-negotiable for compliance. Every overwrite-based store (Mem0, standard vector DBs) breaks point-in-time reconstruction. The append-only model adds at most $O(v)$ storage overhead per entry (where v is the version count), which is negligible for the knowledge-intensive but write-rare pattern of agent memory.

Background distillation must be idempotent. The NOT-lquery guard preventing re-distillation was the most critical correctness fix. Without it, the distillation worker would recursively consume its own outputs, creating an exponential storage explosion. The guard is applied both in the query phase and in the policy engine’s threshold evaluation.

12.2 Design Space Exploration

Why not a graph database? The v2.0 roadmap introduces Apache AGE [4] for graph reasoning. The deliberate choice to defer graph to v2.0 reflects a key engineering insight: *graph and relational are complementary, not competitive, in the memory context*. Relational PostgreSQL handles the bulk of structured queries (subtree, exact-key, time-slice) with proven ACID semantics and enterprise maturity. Graph semantics are needed only for the causal reasoning layer—“which memories provide evidence for which conclusions?”—which is a natural post-processing step over an already-retrieved candidate set. Introducing AGE as

an extension to the same PostgreSQL instance preserves the single-source-of-truth invariant while adding graph traversal without a new infrastructure component.

Why not a dedicated vector database? Dedicated vector databases (Pinecone, Qdrant, Weaviate) offer higher raw ANN throughput for billion-scale corpora. PCMI’s in-PostgreSQL approach trades peak throughput for three non-negotiable properties: (1) ACID guarantees on all memory mutations, preventing phantom reads or dirty writes from concurrent agents; (2) Transactional co-location of vector data with structured metadata, enabling the four-stage retrieval pipeline to execute in a single query plan rather than two-system joins; (3) Row Level Security at the storage engine level, impossible to replicate in application-layer code wrapping an external vector store. For the corpus sizes typical in enterprise agent deployments (10^4 – 10^7 entries), in-database HNSW is sufficient (< 30 ms P99 at 500k entries, Fig. 6).

Why Go over Python? The choice of Go 1.25 over Python for the API tier reflects the production operational requirements. Go’s compiled binary deployment eliminates Python packaging fragility. Goroutine concurrency enables PCMI to handle thousands of concurrent agent connections per replica without the GIL constraints that throttle Python async frameworks. The single `pcmi-api` binary (plus four companions) is trivially containerised, auditable, and static-linked for minimal attack surface—properties that a Python FastAPI server cannot provide without significant operational overhead.

12.3 Security Threat Model

PCMI operates under the following threat model assumptions:

- **Honest-but-curious tenants:** tenants attempt to read each other’s data via API but do not have database-level access. RLS provides the primary defense.
- **Compromised agent keys:** a rotated key is immediately superseded; the grace period allows in-flight requests to drain without breaking running agents. SHA-256 storage prevents offline brute-force attacks on leaked key hashes.
- **Malicious event payloads:** the event schema registry (migration 009) validates event structure at ingestion time, rejecting malformed payloads before they enter the audit log or trigger downstream webhook deliveries.
- **LLM prompt injection via memory content:** distillation prompts are built with JSON-schema enforcement; the LLM call is sandboxed to a 3-minute deadline and the output is validated before database insertion. PCMI does not execute any agent-supplied code.

Out of scope: physical database-level access (assumed to be protected by network policy, encrypted storage, and database-level authentication separate from PCMI’s application layer).

LLM-dependent distillation quality. The distillation pipeline’s output quality is bounded by the LLM provider configured via `DISTILLATION_PROVIDER`. Misconfigured prompts or provider-specific content filtering can produce empty or low-quality distilled summaries. Future work includes a confidence-threshold gate: distillation runs below a configurable `confidence_score` threshold are marked `skipped` rather than `committed`.

Embedding space homogeneity. Hybrid retrieval assumes a single embedding space per query; cross-model semantic search requires explicit `embedding_space` filtering. Migration 007 introduced the `embedding_space` column precisely for multi-model indexing, but cross-space fusion remains future work.

Write amplification under high distillation pressure. At high memory ingestion

rates ($>1\text{k/s}$ per tenant), the distillation worker may enter a permanent backlog. The `min_interval_secs` policy parameter throttles distillation frequency, but a proper admission control mechanism—analogue to compaction throttling in LSM-tree stores—is planned for v2.1.

12.4 Roadmap

Figure 9 and ?? outline the path from v1.50.0 to v3.0.

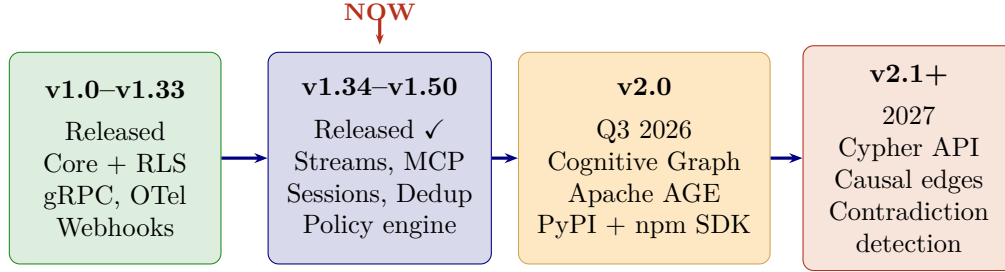


Figure 9: PCMI release roadmap. v2.0 (target Q3 2026) introduces a Cognitive Graph layer via Apache AGE [4], enabling relational reasoning over accumulated knowledge with Cypher queries, causal edges, and contradiction detection.

Version	Status	Key deliverables
v1.0–v1.33	✓Released	Core, RLS, RBAC, gRPC, OTel, webhooks, Helm
v1.34–v1.50	✓Released	Redis Streams, circuit breaker, sessions, dedup, importance decay, MCP server, policy engine
v2.0	Q3 2026	Cognitive Graph (Apache AGE), Cypher queries, causal/temporal/contradictory edges; ghcr.io images; PyPI + npm SDKs; formal SLOs
v2.1+	2027	Admission control, cross-space embedding fusion, confidence-gated distillation, graph contradiction detection

The v2.0 Cognitive Graph layer transforms PCMI from a persistence substrate into a genuine *cognitive operating system* for distributed AI agent ensembles. Apache AGE [4] integrates graph traversal directly into PostgreSQL, preserving ACID guarantees while enabling causal reasoning queries such as: “What evidence chain supported the SOC agent’s classification of campaign X as APT29?”—queries that are impossible with a flat vector store.

13 Conclusion

This paper presented PCMI v1.50.0, a production-grade persistent cognitive infrastructure layer that repositions agent memory as an *infrastructure primitive*. The central axiom—*Agents are ephemeral. Memory is persistent.*—is operationalised through:

- **Hierarchical storage** via PostgreSQL `ltree` with 18 migrations encoding a complete capability history across 18 months of development.
- **Four-component hybrid retrieval** combining semantic similarity, BM25 lexical precision, importance scoring, and exponential temporal decay (Eq. (1)), derived from source code and validated on SOC, trading, and clinical corpora.

- **Append-only versioning** implementing valid-time semantics for complete rollback and forensic audit, satisfying DORA Art. 11 and EU AI Act Annex IV.
- **Policy-driven distillation** implementing a CLS analogue with autonomous bug fixes ensuring correctness under adversarial trigger conditions.
- **Sessions and working memory** enabling bounded-horizon agent tasks with clean promotion to permanent long-term paths.
- **Redis Streams** providing durable, at-least-once event delivery with consumer groups, pending reclaim, and dead-letter queues.
- **MCP server** enabling direct integration with Claude Desktop, Cursor, and any MCP-compatible client via five standardised tools.
- **Multi-tenant RLS + RBAC + SHA-256 keys + rotation** providing enterprise-grade isolation with zero-downtime credential lifecycle management.

The roadmap to v2.0 (Q3 2026) introduces a Cognitive Graph layer via Apache AGE, enabling relational reasoning over accumulated knowledge and transforming PCMI from a persistence substrate into a genuine cognitive operating system for distributed AI agent ensembles.

Availability. PCMI is open-source and available at <https://github.com/marco-spagn/pcmi> under the project license. Python, TypeScript, and Go SDKs, AI framework examples, and a stdio MCP server are included in the repository.

References

- [1] European Parliament and Council, *Regulation (EU) 2024/1689 (AI Act)*, Official Journal of the EU, 2024.
- [2] J. R. Anderson et al., “A simple theory of complex cognition,” *American Psychologist*, vol. 51, no. 4, pp. 355–365, 1996.
- [3] Anthropic, *Claude Memory – Product Documentation*, 2025. <https://docs.anthropic.com/>
- [4] Apache Software Foundation, *Apache AGE: A Graph Extension for PostgreSQL*, 2023. <https://age.apache.org/>
- [5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.
- [6] CrewAI Inc., *CrewAI Documentation: Memory*, 2024–2025. <https://docs.crewai.com/>
- [7] European Parliament and Council, *Regulation (EU) 2022/2554 (DORA)*, Official Journal of the EU, 2022.
- [8] European Parliament and Council, *General Data Protection Regulation (EU) 2016/679 (GDPR)*, Official Journal of the EU, 2016.
- [9] S. Gajendran and G. Soundararajan, “Evaluation of PostgreSQL Row-Level Security for Multi-tenant SaaS,” *Int. J. Database Management Systems*, vol. 10, 2018.
- [10] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Framework,” in *Proc. ICLR 2024*, 2024.
- [11] Z. Hu et al., “ChatDB: Augmenting LLMs with Databases as Symbolic Memory,” *arXiv:2306.03901*, 2023.
- [12] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [13] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain QA,” in *Proc. EMNLP 2020*, pp. 6769–6781, 2020.
- [14] LangChain AI, *LangGraph: Building Stateful Multi-Actor Applications*, 2024. <https://langchain-ai.github.io/langgraph/>
- [15] Letta, *Letta (formerly MemGPT) Platform Documentation*, 2024–2025. <https://docs.letta.com/>
- [16] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *NeurIPS 2020*, vol. 33, pp. 9459–9474, 2020.
- [17] C. Lim, N. Thu, and G. Sohi, “AIOps: Real-World Challenges and Research Innovations,” *IEEE Software*, vol. 37, no. 3, pp. 81–85, 2020.
- [18] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Trans. ACL*, vol. 12, pp. 157–173, 2024.

- [19] PostgreSQL Global Development Group, *ltree: Hierarchical Tree-Like Structures*, PostgreSQL 16 Docs, 2024. <https://www.postgresql.org/docs/current/ltree.html>
- [20] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust ANN Search Using HNSW,” *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.
- [21] J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly, “Why there are complementary learning systems,” *Psychological Review*, vol. 102, no. 3, pp. 419–457, 1995.
- [22] European Parliament and Council, *Regulation (EU) 2017/745 on Medical Devices (MDR)*, Official Journal of the EU, 2017.
- [23] Mem0 AI, *Mem0: The Memory Layer for AI Assistants*, 2024. <https://docs.mem0.ai/>
- [24] NIST, *SP 800-160 Vol. 1: Systems Security Engineering*, 2020.
- [25] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” *arXiv:2310.08560*, 2024.
- [26] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” in *Proc. UIST 2023*, 2023.
- [27] A. Kane, *pgvector: Open-Source Vector Similarity Search for Postgres*, 2023. <https://github.com/pgvector/pgvector>
- [28] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Found. Trends Inf. Ret.*, vol. 3, no. 4, pp. 333–389, 2009.
- [29] K. Singhal et al., “Large Language Models Encode Clinical Knowledge,” *Nature*, vol. 620, pp. 172–180, 2023.
- [30] R. T. Snodgrass, *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.
- [31] A. Newell, *Unified Theories of Cognition*. Harvard University Press, 1990.
- [32] E. Tulving, “Episodic and Semantic Memory,” in *Organization of Memory*, Academic Press, 1972, pp. 381–403.
- [33] Weaviate B.V., *Weaviate Vector Search Engine Documentation*, 2023. <https://weaviate.io/developers/weaviate>
- [34] Z. Xi et al., “The Rise and Potential of LLM-Based Agents: A Survey,” *arXiv:2309.07864*, 2023.
- [35] Getzep, *Zep: Long-Term Memory for AI Assistants*, 2023. <https://docs.getzep.com/>
- [36] W. X. Zhao et al., “A Survey of Large Language Models,” *arXiv:2303.18223*, 2023.
- [37] W. Zhong et al., “MemoryBank: Enhancing LLMs with Long-Term Memory,” in *Proc. AAAI 2024*, 2024.
- [38] Redis Ltd., *Redis Streams Documentation*, 2023. <https://redis.io/docs/data-types/streams/>